

SelectItems en JSF



SelectItems en JSF

1. ¿Qué son los componentes de tipo Select Items en JSF?

En JSF, los componentes de tipo **select items** son usados para construir listas desplegables, menús y opciones seleccionables en formularios. Estos componentes permiten al usuario elegir uno o varios valores de una lista predefinida.

JSF ofrece varios componentes para manejar opciones seleccionables:

- `<h:selectOneMenu>`: Permite seleccionar un único valor.
- `<h:selectManyCheckbox>`: Permite seleccionar múltiples valores mediante casillas de verificación.
- `<h:selectOneRadio>`: Permite seleccionar un único valor mediante botones de opción.

Los valores para estas listas pueden ser dinámicos (extraídos de un bean) o estáticos (definidos directamente en la página).

2. Componentes más comunes

1. `<h:selectOneMenu>`

Este componente renderiza una lista desplegable para que el usuario seleccione un solo valor.

2. `<h:selectManyCheckbox>`
Renderiza múltiples casillas de verificación para que el usuario pueda seleccionar varios valores.
3. `<h:selectOneRadio>`
Renderiza botones de opción que permiten seleccionar un solo valor.

Proyecto Completo

1. Crear el proyecto en NetBeans

1. Abre NetBeans y selecciona **File > New Project**.
2. En la sección **Maven**, elige **Jakarta EE with Maven** y haz clic en **Next**.
3. Configura los valores del proyecto: SelectItemsJSF
4. Selecciona **Jakarta EE** como perfil.
5. Configura GlassFish como servidor de aplicaciones y haz clic en **Finish**.

2. Crear los recursos paso a paso

2.1 Clase modelo: Colonia.java

```
package beans.model;

public class Colonia {
    private int coloniaId;
    private String nombreColonia;
    private int codigoPostal;

    public Colonia(int coloniaId, String nombreColonia, int codigoPostal) {
        this.coloniaId = coloniaId;
        this.nombreColonia = nombreColonia;
        this.codigoPostal = codigoPostal;
    }

    public Colonia() {
    }

    public int getColoniaId() {
        return coloniaId;
    }

    public void setColoniaId(int coloniaId) {
        this.coloniaId = coloniaId;
    }

    public String getNombreColonia() {
```

```
        return nombreColonia;
    }

    public void setNombreColonia(String nombreColonia) {
        this.nombreColonia = nombreColonia;
    }

    public int getCodigoPostal() {
        return codigoPostal;
    }

    public void setCodigoPostal(int codigoPostal) {
        this.codigoPostal = codigoPostal;
    }
}
```

2.2 Clase Helper: ColoniaHelper.java

```
package beans.helper;

import beans.model.Colonia;
import java.util.ArrayList;
import java.util.List;
import jakarta.enterprise.context.RequestScoped;
import jakarta.faces.model.SelectItem;
import jakarta.inject.Named;

@RequestScoped
@Named
public class ColoniaHelper {

    public List<Colonia> getColonias(){
        List<Colonia> colonias = new ArrayList<>();

        int coloniaId = 1;
        Colonia colonia = new Colonia(coloniaId++, "Napoles", 3810);
        colonias.add(colonia);

        colonia = new Colonia(coloniaId++, "Polanco", 11530 );
        colonias.add(colonia);

        colonia = new Colonia(coloniaId++, "Del Valle Centro", 3100 );
        colonias.add(colonia);
    }
}
```

```
        return colonias;
    }

    public int getColonialdPorNombre(String nombreColonial){
        int coloniald = 0;
        List<Colonial> colonias = this.getColonias();

        for(Colonial colonial: colonias){
            if(colonial.getNombreColonial().equals(nombreColonial)){
                coloniald = colonial.getColoniald();
                break;
            }
        }
        return coloniald;
    }

    public int getColonialdPorCP(int codigoPostal){
        int coloniald = 0;
        List<Colonial> colonias = this.getColonias();
        for(Colonial colonial: colonias){
            if(colonial.getCodigoPostal() == codigoPostal){
                coloniald = colonial.getColoniald();
                break;
            }
        }
        return coloniald;
    }

    public List<SelectItem> getSelectItems(){
        List<SelectItem> selectItems = new ArrayList<>();
        List<Colonial> colonias = this.getColonias();
        for(Colonial colonial: colonias){
            SelectItem selectItem = new SelectItem(colonial.getColoniald(),
                colonial.getNombreColonial());
            selectItems.add(selectItem);
        }
        return selectItems;
    }
}
```

Explicación:

- Esta clase contiene métodos para obtener las colonias y convertirlas en elementos seleccionables (SelectItem).
- Los valores se usan dinámicamente en el componente h:selectOneMenu.

2.3 Backing Bean: vacanteForm.java

```
package beans.backing;

import beans.helper.ColoniaHelper;
import beans.model.Candidato;
import jakarta.enterprise.context.RequestScoped;
import jakarta.faces.application.FacesMessage;
import jakarta.faces.component.UIInput;
import jakarta.faces.component.UIViewRoot;
import jakarta.faces.context.FacesContext;
import jakarta.faces.event.ActionEvent;
import jakarta.faces.event.ValueChangeEvent;
import jakarta.inject.Inject;
import jakarta.inject.Named;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Named
@RequestScoped
public class VacanteForm {

    private Logger log = LoggerFactory.getLogger(VacanteForm.class);

    private boolean comentarioEnviado;

    @Inject
    private Candidato candidato;

    @Inject
    private ColoniaHelper coloniaHelper;

    public VacanteForm() {
        log.info("Creando el objeto VacanteForm");
    }

    public void setCandidato(Candidato candidato) {
        this.candidato = candidato;
    }

    public String enviar() {
        if ("Juan".equals(candidato.getNombre()) &&
            "Perez".equals(candidato.getApellido())) {
            String msg = "Gracias Juan Perez, te has registrado correctamente.";
            FacesMessage facesMessage =
```

```
        new FacesMessage(FacesMessage.SEVERITY_ERROR, msg, null);
        FacesContext.getCurrentInstance().addMessage(null, facesMessage);
        log.info("Usuario registrado correctamente");
        return "index"; // Redirige a la misma página
    }
    log.info("Datos de usuario inválidos");
    return "fallo"; // Redirige a una página de éxito
}

public void codigoPostalListener(ValueChangeEvent valueChangeEvent) {
    FacesContext facesContext = FacesContext.getCurrentInstance();
    UIViewRoot uiViewRoot = facesContext.getViewRoot();
    int nuevoCodigoPostal =
        ((Long) valueChangeEvent.getNewValue()).intValue();
    UIInput coloniaIdInputText =
        (UIInput) uiViewRoot.findComponent("vacanteForm:coloniaId");
    int coloniaId = this.coloniaHelper.getColoniaIdPorCP(nuevoCodigoPostal);
    coloniaIdInputText.setValue(coloniaId);
    coloniaIdInputText.setSubmittedValue(coloniaId);

    UIInput ciudadInputText =
        (UIInput) uiViewRoot.findComponent("vacanteForm:ciudad");
    String nuevaCiudad = "Ciudad de Mexico";
    ciudadInputText.setValue(nuevaCiudad);
    ciudadInputText.setSubmittedValue(nuevaCiudad);

    facesContext.renderResponse();
}

public void ocultarComentario(ActionEvent actionEvent) {
    this.comentarioEnviado = !this.comentarioEnviado;
}

public boolean isComentarioEnviado() {
    return comentarioEnviado;
}

public void setComentarioEnviado(boolean comentarioEnviado) {
    this.comentarioEnviado = comentarioEnviado;
}

public ColoniaHelper getColoniaHelper() {
    return coloniaHelper;
}
```

```

    public void setColoniaHelper(ColoniaHelper coloniaHelper) {
        this.coloniaHelper = coloniaHelper;
    }
}

```

Mejores prácticas implementadas:

- Uso de `@Inject` para inyectar dependencias (Candidato y ColoniaHelper).
- Uso de SLF4J para registrar logs de eventos importantes.

2.4 Archivo `index.xhtml`

1. Crea el archivo `index.xhtml` en `src/main/webapp`.

```

<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="jakarta.faces.html"
      xmlns:f="jakarta.faces.core">
    <h:head>
        <title>Bolsa de Trabajo V8</title>
    </h:head>
    <h:body>
        <h:form id="vacanteForm">
            <h:messages globalOnly="true" ></h:messages>

            <table>
                <tr>
                    <td><h:outputLabel for="nombre"
                        value="#{msgs['vacanteForm.nombre']}"></h:outputLabel></td>
                    <td><h:inputText id="nombre" required="true"
                        value="#{candidato.nombre}"></h:inputText></td>
                    <td><h:message for="nombre"></h:message></td>
                </tr>
                <tr>
                    <td><h:outputLabel for="apellido"
                        value="#{msgs['vacanteForm.apellido']}"></h:outputLabel></td>
                    <td><h:inputText id="apellido" required="true"
                        value="#{candidato.apellido}"></h:inputText></td>
                    <td><h:message for="apellido"></h:message></td>
                </tr>
                <tr>
                    <td><h:outputLabel for="salarioDeseado"
                        value="#{msgs['vacanteForm.salarioDeseado']}"></td>
                    <td><h:inputText id="salarioDeseado" required="true"

```

```

        value="#{candidato.salarioDeseado}">
        <f:validateLongRange minimum="500" maximum="5000"/>
    </h:inputText>
</td>
<td><h:message for="salarioDeseado"></h:message></td>
</tr>
<tr>
    <td><h:outputLabel for="fechaNacimiento"
value="#{msgs['vacanteForm.fechaNacimiento']}"></h:outputLabel></td>
    <td><h:inputText id="fechaNacimiento" required="true"
        value="#{candidato.fechaNacimiento}">
        <f:convertDateTime pattern="dd/MM/yyyy"></f:convertDateTime>
    </h:inputText>
    </td>
    <td><h:message for="fechaNacimiento"></h:message></td>
</tr>
<tr>
    <td><h:outputLabel for="codigoPostal"
        value="#{msgs['vacanteForm.codigoPostal']}" ></h:outputLabel></td>
    <td><h:inputText converter="jakarta.faces.Integer" id="codigoPostal"
        immediate="true" onchange="this.form.submit()" required="true"
        value="#{candidato.codigoPostal}"
        valueChangeListener="#{vacanteForm.codigoPostalListener}">
        <f:convertNumber integerOnly="true" pattern="00000" />
    </h:inputText></td>
    <td><h:message for="codigoPostal"></h:message></td>
</tr>
<tr>
    <td><h:outputLabel for="coloniaId"
        value="#{msgs['vacanteForm.coloniaId']}" ></h:outputLabel></td>
    <td>
        <h:selectOneMenu id="coloniaId" required="true"
            value="#{candidato.coloniaId}">
            <f:selectItem itemLabel="#{msgs['vacanteForm.seleccione']}"
                itemValue="" />
            <f:selectItems value="#{coloniaHelper.selectItems}" />
        </h:selectOneMenu>
    </td>
    <td><h:message for="coloniaId"></h:message></td>
</tr>
<tr>
    <td><h:outputLabel for="ciudad" value="#{msgs['vacanteForm.ciudad']}"
        ></h:outputLabel></td>
    <td><h:inputText id="ciudad" required="true"
        value="#{candidato.ciudad}"></h:inputText></td>

```

```

        <td><h:message for="ciudad"></h:message></td>
    </tr>
</table>
<table>
    <tr>
        <td>
            <h:commandLink actionListener="#{vacanteForm.ocultarComentario}"
                immediate="true"
                rendered="#{!vacanteForm.comentarioEnviado}"
                value="#{msgs['vacanteForm.mostrar']}" />
            <h:commandLink actionListener="#{vacanteForm.ocultarComentario}"
                immediate="true"
                rendered="#{vacanteForm.comentarioEnviado}"
                value="#{msgs['vacanteForm.ocultar']}" />
        </td>
    </tr>
</table>
<h:panelGroup rendered="#{vacanteForm.comentarioEnviado}">
    <table>
        <tr>
            <td><h:outputLabel for="comentario"
                value="#{msgs['vacanteForm.comentario']}" /></td>
        </tr>
        <tr>
            <td><h:inputTextarea id="comentario" cols="50" rows="10"
                value="#{candidato.comentario}" /></td>
        </tr>
    </table>
</h:panelGroup>
<h:commandButton action="#{vacanteForm.enviar}"
    value="#{msgs['vacanteForm.enviar']}" />
</h:form>
</h:body>
</html>

```

Explicación del componente `selectOneMenu`:

- El componente utiliza el método `getSelectItems()` de `ColoniaHelper` para generar las opciones dinámicas.
- Cuando el usuario proporciona un código postal, el método `codigoPostalListener` ajusta automáticamente la colonia y la ciudad.

3. Estructura recomendada del proyecto

Tu proyecto debería verse así:

```
src/main/java/
├── beans/
│   ├── backing/
│   │   └── VacanteForm.java
│   ├── helper/
│   │   └── ColoniaHelper.java
│   └── model/
│       └── Colonia.java
src/main/webapp/
├── WEB-INF/
│   ├── web.xml
│   └── faces-config.xml
└── index.xhtml
src/main/resources/
├── mensajes_es.properties
└── mensajes_en.properties
```

4. Despliegue de la aplicación

1. Limpia y construye el proyecto: **Run > Clean and Build Project.**
2. Ejecuta el proyecto: **Run > Run Project.**
3. Accede a la aplicación desde el navegador

5. Explicación de la aplicación

Esta aplicación incluye un formulario interactivo que:

1. Permite al usuario ingresar datos personales, como nombre, apellido y salario deseado.
2. Utiliza un campo `h:selectOneMenu` para seleccionar una colonia. Las opciones se generan dinámicamente desde el bean `ColoniaHelper`.
3. Cambia automáticamente la colonia y la ciudad al ingresar un código postal válido.

Sigue adelante con tu aprendizaje 🚀 , ¡el esfuerzo vale la pena!

Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)